SKILL-12

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   int fd[2];
   pid_t pid1, pid2;

   if (pipe(fd) == -1)
   {
      perror("pipe");
      return 1;
   }

   /* First child: execute ls */
   pid1 = fork();

   if (pid1 == -1)
   {
      perror("fork");
      close(fd[0]);
      close(fd[1]);
      return 1;
   }
```

```
if (pid1 == 0)
{
   close(fd[0]);  /* Child does not read */

   if (dup2(fd[1], STDOUT_FILENO) == -1)
   {
      perror("dup2");
      _exit(1);
   }

   close(fd[1]);

   execlp("ls", "ls", (char *)NULL);

   perror("execlp ls");
   _exit(127);
}

/* Second child: execute wc -l */
pid2 = fork();

if (pid2 == -1)
{
   perror("fork");
   close(fd[0]);
   close(fd[1]);
   waitpid(pid1, NULL, 0);
```

```
    return 1;
}

if (pid2 == 0)
{
    close(fd[1]);  /* Child does not write */

    if (dup2(fd[0], STDIN_FILENO) == -1)
    {
        perror("dup2");
        _exit(1);
    }

    close(fd[0]);

    execlp("wc", "wc", "-l", (char *)NULL);

    perror("execlp wc");
    _exit(127);
}

/* Parent does not use the pipe */
close(fd[0]);
close(fd[1]);

/* Synchronize with both children */
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
```

```
    printf("Pipeline completed.\n");

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano single_pipe.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -o single_pipe single_
pipe.c
snigdha@Snigdha:~/quotes_practical$ ./single_pipe
65
Pipeline completed.
snigdha@Snigdha:~/quotes_practical$ ls | wc -l
65
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10

struct Command
{
    char *args[10];
};

int main(void)
{
    struct Command commands[] = {
        {{"printf", "pear\napple\npear\nbanana\n", NULL}},
```

```
    {{"sort", NULL}},
    {{"uniq", NULL}},
    {{"wc", "-l", NULL}}
};

int n = (int)(sizeof(commands) / sizeof(commands[0]));
int pipes[MAX_COMMANDS - 1][2];
pid_t pids[MAX_COMMANDS];

if (n < 1 || n > MAX_COMMANDS)
{
    fprintf(stderr, "Invalid number of commands\n");
    return 1;
}

/* Create all pipes before launching processes */
for (int i = 0; i < n - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe");

        for (int j = 0; j < i; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }
```

```
      return 1;
    }
}

/* Launch one process for each command */
int launched = 0;

for (int i = 0; i < n; i++)
{
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        break;
    }

    if (pid == 0)
    {
        /* Read from the previous pipe */
        if (i > 0)
        {
            if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
            {
                perror("dup2 input");
                _exit(1);
            }
        }
```

```
      /* Write to the next pipe */
      if (i < n - 1)
      {
         if (dup2(pipes[i][1], STDOUT_FILENO) == -1)
         {
            perror("dup2 output");
            _exit(1);
         }
      }

      /* Close all original pipe descriptors */
      for (int j = 0; j < n - 1; j++)
      {
         close(pipes[j][0]);
         close(pipes[j][1]);
      }

      execvp(commands[i].args[0], commands[i].args);

      perror(commands[i].args[0]);
      _exit(127);
   }

   pids[launched++] = pid;
}

/* Parent closes all pipe descriptors */
```

```
for (int i = 0; i < n - 1; i++)
{
   close(pipes[i][0]);
   close(pipes[i][1]);
}

/* Reap every process that was successfully launched */
int failed = (launched != n);

for (int i = 0; i < launched; i++)
{
   int status;

   if (waitpid(pids[i], &status, 0) == -1)
   {
      perror("waitpid");
      failed = 1;
   }
   else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0)
   {
      failed = 1;
   }
}

if (failed)
{
   fprintf(stderr, "Pipeline failed or did not launch completely.\n");
   return 1;
```

```
    }

    fprintf(stderr, "Multi-stage pipeline completed.\n");
    return 0;
}
```

